OpenClaw Skills Development

OpenClaw Skills Development

1. Course Content
    - Course Overview
    - Learning Objectives
2. Skills Introduction
3. Robot Factory-Preset Skills
4. Agent Skill Management
5. Custom Skill Development
    - 5.1 How to Develop Your Own Skill
    - 5.2 Creating a Skill
    - 5.3 Skill Document Format
    - 5.4 Writing Skill Content
    - 5.5 Enabling Skills in OpenClaw

1. Course Content

Course Overview

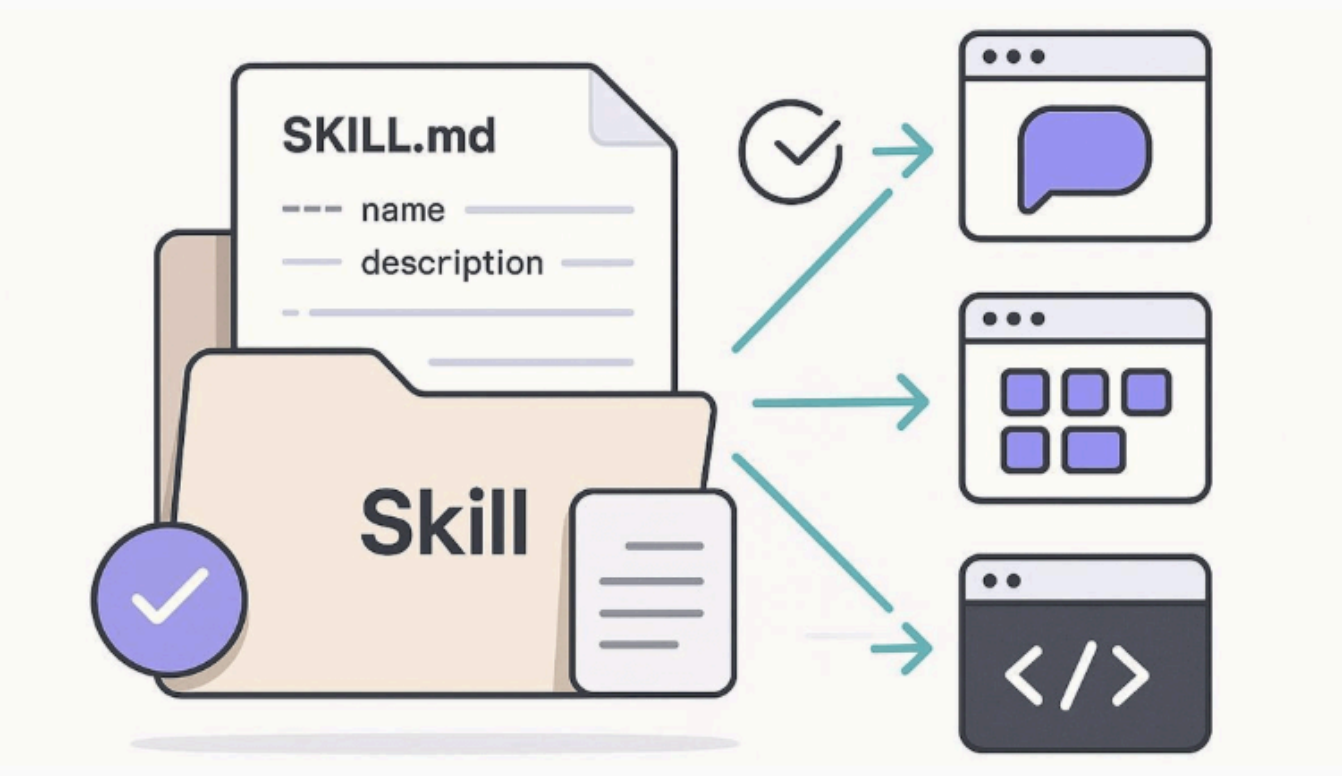
- OpenClaw Skills is a set of advanced task instruction sets for developers, which orchestrates multiple MCP Tools into a complete workflow according to business logic, guiding the AI model to call the right tools at the right time to complete complex tasks
- The robot comes pre-installed with 5 core skills: robotic arm pick-and-place, waste sorting, navigation and localization, visual tracking, and comprehensive tasks, covering the robot's main application scenarios

Learning Objectives

1. Understand the concept, principles, and role of OpenClaw Skills
2. Become familiar with the functions and usage of the robot's factory-preinstalled core skills
3. Master the operations for enabling and disabling skills in the Agent

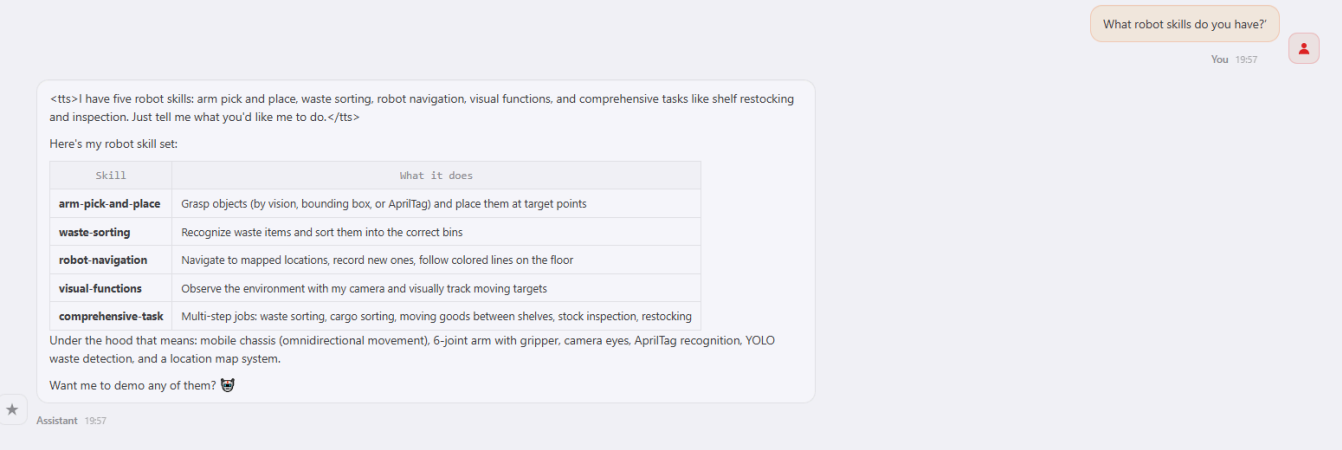
2. Skills Introduction

- Skills is a **task-level workflow instruction set**
- The AI model follows the workflow defined in the skill file, calling MCP Tools step by step, completing the full closed loop from environment perception → decision planning → action execution → result verification
- Skill files are stored in the `$HOME/.openclaw/workspace/skills` directory. Each skill corresponds to a subdirectory, and the `SKILL.md` file within defines the complete skill instructions



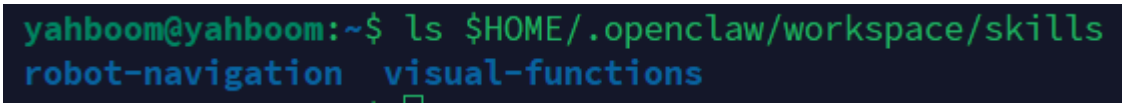
3. Robot Factory-Preset Skills

- You can directly ask OpenClaw in conversation which skills it currently has



- Skills folder path:

```
$HOME/.openclaw/workspace/skills
```



The robot comes pre-installed with the following skills:

- 🦾 **① arm-pick-and-place (Robotic Arm Pick and Place)**
  - **Function**: Use the robotic arm to complete object picking and placing operations
  - **Trigger Keywords**: grasp / pick / place / grab / move / pick up
  - **Core Tools**: Pick / Place / SeeWhat / AdjustCameraView / GetPlacePoint / GetBbox
  - **Workflow**: Observe the frame → locate the target → perform grasping → check the result → place at the target point

♻️ ② waste-sorting (Waste Sorting)

- **Function**: Recognize the waste type through vision and sort it into the corresponding trash bin, navigating to each trash bin to complete disposal
- **Trigger Keywords**: sorting / grip / pick up / pick up waste / waste sorting / waste observation
- **Core Tools**: GetWasteRecognitionResults / GraspWaste / Navigation / PlaceWaste
- **Workflow**: Get the recognition result → grasp the waste → navigate to the corresponding trash bin → place into the trash bin

🧭 ③ robot-navigation (Navigation and Localization)

- **Function**: Navigate to the target position according to route rules on the map, supporting recording and reading of map points
- **Core Tools**: Navigation / GetMapMapping / RecordMapLocation
- **Workflow**: Get the map mapping → find the target location → execute navigation (supports 3 retries)
- **Additional Features**: Use RecordMapLocation to record the current position into the map mapping

👁️ ④ visual-functions (Visual Functions)

- **Function**: Observe the environment and perform visual tracking of target objects
- **Trigger Keywords**: Observe the environment / Visual tracking target
- **Core Tools**: SeeWhat / AdjustCameraView / GetBbox / TargetTrack
- **Workflow**: Get the frame → locate the target → call TargetTrack to track the target

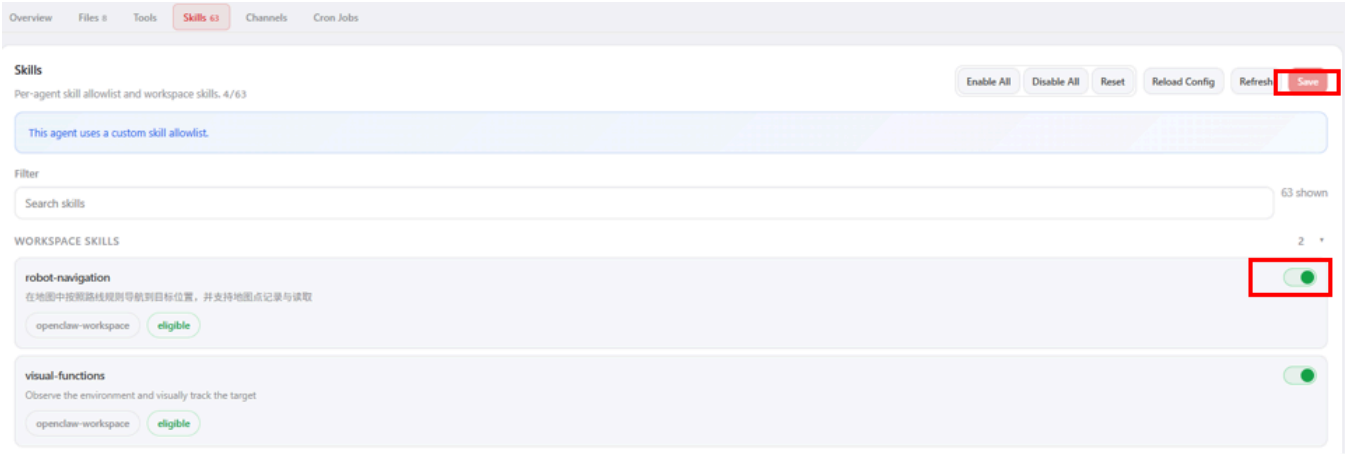
⚙️ ⑤ comprehensive-task (Comprehensive Task)

- **Function**: Covers various comprehensive scenarios such as waste sorting, cargo sorting, transport between shelves, and cargo inspection and restocking
- **Trigger Keywords**: cargo sorting / restocking / waste handling / shelf operation / cargo inspection and restocking
- **Sub-flows**: Waste sorting → refer to waste-sorting; cargo sorting → navigation + grasping + transport + placement; shelf transport → navigation → grasping → navigation → placement; cargo inspection and restocking → inspection → navigation → grasping → return → placement

The above skill files are stored in the `$HOME/.openclaw/workspace/skills` directory, with each skill in its own subdirectory. You can directly view or modify the `SKILL.md` file inside.

4. Agent Skill Management

- Log in to OpenClaw's WebChat, then click **Agent** → **Skills**
- Under the **Workspace Skills** tab, you can see the factory-preset skills in the workspace. If you need to enable or disable an individual skill, you can click the toggle switch, then click **Save** to apply



5. Custom Skill Development

5.1 How to Develop Your Own Skill

- A custom Skill is essentially writing a `SKILL.md` file that conforms to the format specification and placing it in the `$HOME/.openclaw/workspace/skills` directory
- Developing a skill requires completing the following steps:
    1. **Create the skill directory**: Create a new subdirectory under the `skills` directory; the directory name is the skill name (lowercase letters with hyphens recommended)
    2. **Write SKILL.md**: Create a `SKILL.md` file in the subdirectory and write the skill content according to the format
    3. **Restart OpenClaw**: Restart the OpenClaw gateway to activate the new skill
    4. **Enable in the Agent**: Enable the new skill in WebChat's Agent Skills management

5.2 Creating a Skill

Create the skill directory and file

```
mkdir -p $HOME/.openclaw/workspace/skills/<my-custom-skill>
touch $HOME/.openclaw/workspace/skills/<my-custom-skill>/SKILL.md
```

5.3 Skill Document Format

Each `SKILL.md` file consists of two parts:

YAML Meta Header

```
---
name: my-custom-skill          # Skill name (unique identifier)
description: Skill function description       # Brief description of the skill's purpose
version: 1.0.0                 # Version number
author: "Your Name"             # Author information
trigger_keywords:              # Trigger keywords (the AI calls this skill based on these
words)
  - keyword1
  - keyword2
---
```

Parameter Description:

- `name`: Skill name, used as the unique identifier; the lowercase + hyphen format is recommended (e.g., `my-custom-skill`). OpenClaw identifies and loads the skill by this name
- `description`: Skill function description, briefly explaining the skill's purpose; the AI model uses this description to judge when to call the skill
- `version`: Version number, following the semantic versioning convention (e.g., `1.0.0`), used to track the iterative updates of the skill
- `author`: Author information, identifying the developer of the skill
- `trigger_keywords`: Trigger keyword list; the AI model automatically matches and calls the corresponding skill based on keywords in the user input. It is recommended to include both Chinese and English keywords, covering the skill's main operation scenarios

The body section uses Markdown format to write the skill workflow, usually containing:

- **Related Tools**: Lists the MCP Tools required by this skill
- **Rules**: Any necessary constraints and rules
- **Workflow**: Defines the operation flow step by step, including specific command line examples
- **Error Handling**: Handling solutions for common errors

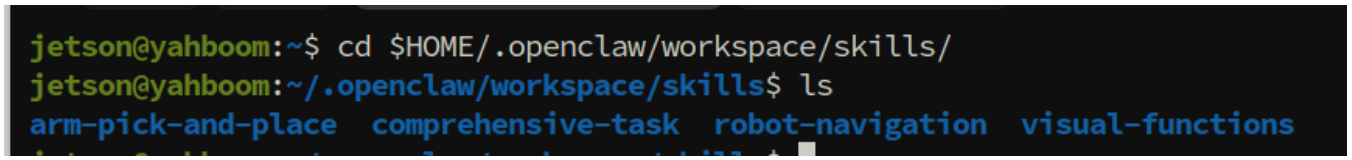
5.4 Writing Skill Content

When writing skill content, follow these principles:

- **Clear step division**: Break the task into clear steps (Step1 / Step2 / ...); each step should have a clear execution goal and judgment criteria
- **Complete tool call examples**: For each MCP Tool that needs to be called in a step, provide the complete command format and parameter description
- **Error handling mechanism**: Provide clear handling strategies for possible error situations (such as target not existing, tool call failure, etc.)
- **Conditional judgment and loops**: Steps that require looping operations should have clear termination

conditions, and steps requiring conditional judgment should provide judgment criteria

> When writing, you can refer to the factory-preset skill's `SKILL.md` file as a template, path:
>
> ```
> $HOME/.openclaw/workspace/skills/
> ```



5.5 Enabling Skills in OpenClaw

- Refer to **4. Agent Skill Management** to enable and disable skills